Equazioni di ricorrenza

Materia	Algoritmi
Anno	Secondo Anno
Data	@October 10, 2024
Studiato	Done

Approccio divide and conquer

Il metodo **divide and conquer** (o *divide et impera*) è una strategia ricorsiva per progettare algoritmi asintoticamente efficienti.
In ogni passo ricorsivo avvengono tre passaggi caratteristici:

- una fase di **divisione** (divide), in cui il problema viene diviso in sotto-problemi, ossia **istanze più piccole dello stesso problema**;
- una fase di **risoluzione** (conquer), in cui il sotto-problema viene risolto;
- una fase di **combinazione** (combine), in cui combiniamo le soluzioni per ottenere la soluzione del problema principale.

Nel processo di divisione arriviamo alla **recursion bottom** in cui **i sotto-problemi sono istanze del caso base**.

> *Un caso base è tale se è un'istanza del problema che è talmente piccola da poter essere risolta in tempo costante.*

Analisi di un algoritmo ricorsivo: equazioni di ricorrenza

Le equazioni di ricorrenza sono lo strumento naturale per analizzare la complessità degli algoritmi ricorsivi.

> *Un'**equazione di ricorrenza** è una formula che descrive il valore di una funzione in corrispondenza di un argomento x generico*

> *in termini del valore della funzione stessa quando essa assume altri argomenti, in genere di dimensione inferiore.*

Supponiamo che l'algoritmo ricorsivo considerato divida un problema di dimensione n in a sottoproblemi di dimensioni $\frac{n}{b}$.

Diremo quindi che:

$$T(n) = \begin{cases} D(n) + aT(\frac{n}{b}) + C(n) & \forall n \geq n_0 \\ \Theta(1) & \forall n < n_0 \end{cases}$$

Chiameremo $D(n)$ la complessità di **divide**, e $C(n)$ la complessità di **combine**.
Tale equazione utilizza il valore della funzione stessa su un input più piccolo.
In alcuni casi, la formula di ricorrenza è una diseguaglianza. Se il segno della diseguaglianza è
$\leq$, allora otteniamo un **upper bound**, mentre per $\geq$ otteniamo un **lower bound**.

Assumeremo, inoltre, che la **ricorrenza sia algoritmica**.

> *Una ricorrenza $T(n)$ è **algoritmica** se, per ogni soglia $n_0 > 0$ sufficientemente grande valgono le seguenti proprietà:*
>
> - $\forall n < n_0, T(n) = \Theta(1)$;
> - $\forall n \geq n_0$, *ogni cammino di ricorsione termina in un caso base **entro un numero finito di chiamate ricorsive**.*

Calcolare la complessità computazionale di un algoritmo ricorsivo significa risolvere l'equazione di ricorsione ad essa associata.

Metodi per risolvere un'equazione di ricorrenza

Vi sono almeno 4 modi per risolvere un'equazione di ricorsione:

- metodo di **sostituzione**;
- metodo dell'**albero di ricorsione**;
- metodo **master**;

- metodo **akra-bazzi.**

Metodo dell'albero di ricorsione: esempio di merge sort

Il metodo dell'albero di ricorsione permette di formulare una buona ipotesi per la soluzione dell'equazione di ricorrenza.

> *Un **albero di ricorsione** è un albero in cui ogni nodo rappresenta il costo di un sottoproblema di una delle chiamate ricorsive della funzione.*

Generalmente sommeremo i costi di ogni nodo per ottenere il costo di tutti i livelli di ricorsione.

Consideriamo il merge sort. La fase di divide calcola la metà dell'array, quindi calcola $\lceil \frac{n}{2} \rceil$ e $\lfloor \frac{n}{2} \rfloor$. La fase di conquer consiste nel risolvere $a = 2$ problemi di dimensione $\frac{n}{2}$ ($b = 2$). Infine, combine (ossia merge) ha complessità $C(n) = \Theta(n)$.

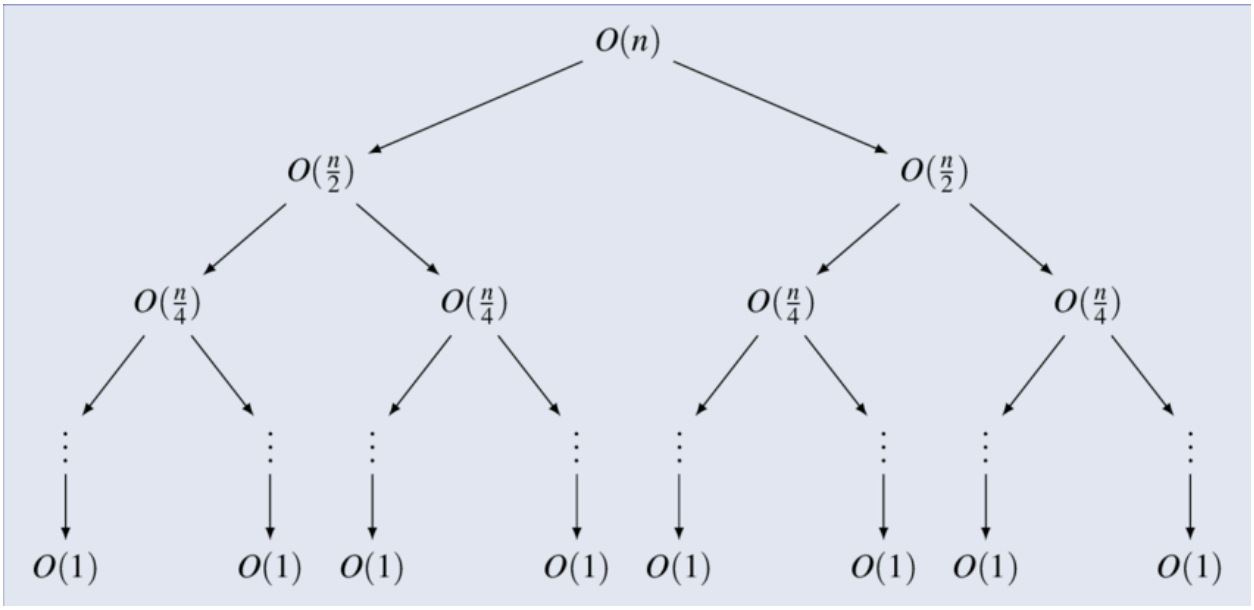
Allora avremo che:

$$T(n) = \Theta(1) + 2T(\frac{n}{2}) + \Theta(n) = 2T(\frac{n}{2}) + \Theta(n)$$

Quindi:

$$T(n) = \begin{cases} 2T(\frac{n}{2}) + \Theta(n) & \forall n \geq 2 \\ \Theta(1) & n = 1 \end{cases}$$

Poiché la differenza tra $\lceil \frac{n}{2} \rceil$ e $\lfloor \frac{n}{2} \rfloor$ è molto piccola, possiamo assumere che avremo sempre partizioni di $\frac{n}{2}$.
Dire
$\Theta(n)$ significa dire che la funzione sarà del tipo $c \cdot n$. Inoltre, poiché la procedura di combine viene eseguita ad ogni chiamata ricorsiva diversa dal caso base, avremo che ogni nodo dell'albero di ricorsione, eccetto le foglie, avrà costo $c \cdot n$.



Con $O(n)$ possiamo in realtà affermare che la funzione è anche $\Omega(n)$, pertanto possiamo scrivere $c \cdot n$.

Ad ogni passo ricorsivo dividiamo l'input per due, pertanto l'albero di ricorsione giungerà ad un punto in cui avremo solamente problemi risolvibili con complessità $\Theta(1) = d$. L'albero di ricorsione avrà altezza $h + 1$, dove $h = \log n$. Infatti, possiamo sicuramente dire che $T(\frac{n}{2^h}) = T(1) \implies \frac{n}{2^h} = 1 \implies n = 2^h \implies h = \log n$.

Nel livello i-esimo dell'albero di ricorsione avremo 2^i nodi di costo $c \cdot \frac{n}{2^i}$, quindi il costo del livello è $c \cdot n$. Al livello h avremo $2^h = 2^{\log n} = n$ nodi di costo d. Quindi $h = \log n$ è il livello delle foglie.

Quindi possiamo dire che:

$$T(n) = \sum_{i=0}^{h-1} cn + dn = h \cdot cn + dn = \log n \cdot cn + dn = n(c \log n + d)$$

Che è $O(n \log n)$. Analogamente si può dimostrare che l'algoritmo è $\Omega(n \log n)$, quindi il merge sort è $\Theta(n \log n)$.

Metodo della sostituzione

Il metodo della sostituzione si basa su due passi:

- **guess**, in cui ipotizziamo la forma della soluzione lasciando costanti generiche;
- **induzione matematica**, in cui verifichiamo la soluzione proposta.

Il nome del metodo deriva dal fatto che la soluzione congetturata viene sostituita al posto della funzione quando l'ipotesi viene applicata a valori più piccoli.

Bisogna sempre **esplicitare le costanti** per evitare errori.

Consideriamo subito un esempio con la seguente ricorrenza:

$$T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + \Theta(n)$$

Supponiamo sia $T(n) = O(n \log n)$. Dobbiamo quindi dimostrare che:

$$\exists c > 0, \exists n_0 : \forall n \geq n_0 \implies T(n) \leq cn \log n$$

Assumiamo come ipotesi induttiva che $T(n) \leq cn \log n$ per ogni $n \geq n_0$, dove c e n_0 sono le costanti che fisseremo una volta aver individuato dei vincoli da rispettare. **Se tale ipotesi è valida, allora possiamo provare che** $T(n) = O(n \log n)$. Sarebbe rischioso, invece, supporre che $T(n) = O(n \log n)$ come ipotesi induttiva.
Poiché ad ogni passo passiamo da
n ad $\frac{n}{2}$, supponiamo che la tesi sia valida per $\frac{n}{2}$ e dimostriamo che è valida per n.
Si ha quindi per ipotesi induttiva che:

$$T(\lfloor \frac{n}{2} \rfloor) \leq c \lfloor \frac{n}{2} \rfloor \log(\lfloor \frac{n}{2} \rfloor)$$

Adesso **sostituiamo il termine** $T(\lfloor \frac{n}{2} \rfloor)$ **con la diseguaglianza trovata**. Otteniamo che:

$$\begin{aligned} T(n) &\leq 2(c \lfloor \frac{n}{2} \rfloor \log(\lfloor \frac{n}{2} \rfloor) + \Theta(n) \\ &\leq cn \log(\frac{n}{2}) + \Theta(n) \\ &= cn \log n - cn \log 2 + \Theta(n) \\ &= cn \log n - cn + \Theta(n) \\ &\leq cn \log n \end{aligned}$$

Possiamo dire che $cn \log n - cn + \Theta(n) \leq cn \log n \iff -cn + \Theta(n) \leq 0$.
Poiché $\Theta(n)$ è l'insieme di funzioni:

$$\Theta(n) = \{\, f(n) : \exists c_1, c_2 \in \mathbb{Q}, \exists n_0 \in \mathbb{N} : \forall n \geq n_0 \implies 0 \leq c_1 n \leq f(n) \leq c_2 n \,\}$$

allora la proposizione è valida se $cn \geq \Theta(n)$, quindi se **la costante c è sufficientemente grande affinché cn domini la funzione anonima $\Theta(n)$**. Poiché ogni funzione $f(n) \in \Theta(n)$ è del tipo $f(n) = dn + q$, si ha che la proposizione vale se prendiamo $c \geq d$.

Rimane solamente da dimostrare che $T(n) \leq cn \log n$ vale anche per i casi base. Tuttavia, **non possiamo semplicemente scegliere** $n_0 = 1$, e per questo motivo sono necessari alcuni accorgimenti:

- la diseguaglianza $T(n) \leq cn \log n$ vale per ogni $n \geq n_0$;
- nel caso in cui da n passiamo direttamente al caso base, avremo che $\frac{n}{2} < n_0$.

Allora **non possiamo utilizzare l'ipotesi induttiva quando da n passiamo subito al caso base**, quindi abbiamo che i casi base dell'induzione sono:

$$n_0 \leq n < 2n_0$$

Per quanto riguarda il caso base, non possiamo prendere $n_0 = 1$ perché avremo $\log 1 = 0$ e abbiamo che $T(n) \leq 0$, ma non è ammissibile che vi sia complessità pari a 0. Dobbiamo supporre quindi che $n_0 = 2$, così che $\log 2 > 0 \implies 2 \log 2 > 0$. I casi base sono quindi $T(2), T(3)$, che hanno tempo costante. Prendiamo allora $c = \max\{\, T(2), T(3) \,\}$ e si ha:

$$T(2) \leq c \leq (2 \log 2)c$$
$$T(3) \leq c \leq (3 \log 3)c$$

e quindi la base dell'induzione è dimostrata.
Abbiamo pertanto dimostrato per induzione che
$T(n) \leq cn \log n$, ossia $T(n) = O(n \log n)$.

Sottrare termini di ordini inferiore per le dimostrazioni

Ci sono casi in cui l'ipotesi è corretta ma i calcoli non risultano corretti nella dimostrazione per induzione.
Il problema in questi casi sta nell'ipotesi induttiva: a volte tale ipotesi **non è abbastanza forte**. Un trucco in questi casi consiste nel **sottrarre un termine di ordine inferiore ala soluzione ipotizzata**. Ad esempio, sia:

$$T(n) = 2T(\frac{n}{2}) + \Theta(1)$$

Supponiamo la tesi sia $T(n) = O(n)$ e proviamo a dimostrare che $T(n) \leq cn$ per ogni $n \geq n_0$ e sostituiamo:

$$T(n) \leq 2(c\frac{n}{2}) + \Theta(1) = cn + \Theta(1)$$

Ma ciò implica che **per qualunque scelta di c vale $T(n) \leq cn$?** Naturalmente no, perché dovremmo dire che $cn + \Theta(1) = cn + d \leq cn$, ma questo non è possibile perché $d \geq 0$.
Allora possiamo aggirare questo problema dicendo che
$T(n) \leq cn - q$. In questo modo abbiamo che $T(n) \leq cn - q \leq cn$.

Evitare gli errori esplicitando le costanti

Data l'equazione di ricorrenza risolta precedentemente:

$$T(n) = 2T(\lfloor\frac{n}{2}\rfloor) + \Theta(n)$$

Potevamo semplicemente concludere che:

$$T(n) \leq 2 \cdot O(\lfloor\frac{n}{2}\rfloor) + \Theta(n) \leq 2 \cdot O(n) + \Theta(n) = O(n)$$

Questa conclusione è **errata** (come abbiamo dimostrato d'altronde), perché la costante c nascosta nella notazione O può subire variazioni. Proviamo a esplicitare la costante ma senza alterare la dimostrazione:

$$T(n) \leq 2(c\lfloor\frac{n}{2}\rfloor) + \Theta(n) \leq cn + \Theta(n) = O(n)$$

E ciò è corretto, ma ora la costante in O dev'essere sicuramente più grande di c, e questo non ci permette di dimostrare che $T(n) \leq cn$.

Esercizio

Sia data l'equazione:

$$T(n) = 3T(\frac{n}{4}) + \Theta(n^2)$$

Vogliamo capire la sua complessità. Possiamo provare con una guess, oppure ipotizzare tramite l'albero.

Dall'equazione di ricorrenza notiamo che scomponiamo il problema di dimensione n in 3 sottoproblemi di dimensione $\frac{n}{4}$. Ogni problema, inoltre, che non è caso base richiede una composizione delle soluzioni di tempo $\Theta(n^2) = cn^2$, per qualche costante c.

Per ogni nodo nell'albero di ricorsione si avranno tre figli. In particolare, la radice avrà valore cn^2. I suoi 3 figli, invece, avranno valore $c(\frac{n}{4})^2$. Notiamo che la somma di questi valori è esattamente $\frac{3}{16}cn^2$. Andando avanti, **si può dedurre che in ogni livello ci sono 3^i nodi, e la somma dei loro costi è** $(\frac{3}{16})^icn^2$.

Se $h+1$ è l'altezza dell'albero, quanti nodi avrà il livello h, ossia il livello delle foglie? Se n è il numero di nodi dell'albero, allora il numero di foglie corrisponde a $3^{\log_4 n} = n^{\log_4 3}$. Poiché ogni foglia corrisponde al caso base, allora il costo del livello h è pari a $\Theta(n^{\log_4 3})$. Allora possiamo stimare che la funzione $T(n)$ è pari a:

$$T(n) = \sum_{i=0}^{\log_4 n - 1} (\frac{3}{16})^icn^2 + \Theta(n^{\log_4 3}) \leq \sum_{i=0}^{\infty} (\frac{3}{16})^icn^2 + \Theta(n^{\log_4 3})$$

Poiché $\frac{3}{16}$ è un numero minore di 1, possiamo approssimare il valore della serie geometrica nel seguente modo:

$$\sum_{i=0}^{\infty} (\frac{3}{16})^i = \frac{1}{1-\frac{3}{16}} = \frac{16}{13}$$

Quindi possiamo dire che $T(n) \leq \frac{16}{13}cn^2 + n^{\log_4 3} = O(n^2)$.

Nota: possiamo anche dire che $T(n) = \Omega(n^2)$, dato che una singola chiamata ricorsiva richiede cn^2 per l'operazione di combine, pertanto si ha $T(n) = \Theta(n^2)$.

Ora che abbiamo una ipotesi - anche ben fondata -, utilizziamo il metodo della sostituzione.
Poiché l'input viene diviso per
4 ad ogni chiamata ricorsiva, la nostra ipotesi induttiva sarà la seguente: per $n \geq n_0$, $\frac{n}{4} \geq n_0$.

$$T(\frac{n}{4}) \leq c(\frac{n}{4})^2$$

dove c è una costante positiva. Sostituendo abbiamo:

$$\begin{aligned} T(n) = 3T(\frac{n}{4}) + dn^2 \leq 3c(\frac{n}{4})^2 + dn^2 = \frac{3}{16}cn^2 + dn^2 \\ \leq cn^2 \iff \frac{13}{16}c \geq d \end{aligned}$$

Da cui deduciamo che $c \geq \frac{16}{13}d$. Rimane solamente da lavorare sul caso base.

Consideriamo $n_0 > 0$ una costante sufficientemente grande da rendere $T(n) = \Theta(1)$ se $n < n_0$. Scegliamo allora d sufficientemente grande in modo tale che domini la funzione $\Theta(n)$, ossia $d \geq \Theta(n)$. Naturalmente si ha $dn^2 \geq d \geq \Theta(n) \implies T(n) \leq dn^2$ anche nel caso base e la ricorrenza è risolta.